

Proof-Driven Multi-Module Python via Agentic TLA⁺ Co-Synthesis

Harry Wang

Eric Shang Nan Chen

May 2026

1 Introduction

LLM-driven program synthesis has converged on a write-first, verify-second loop: emit Python (or Dafny, or Rust), then run a test or a verifier and ask the model to repair. This loop works on isolated functions, but the recent DafnyComp benchmark [1] shows that on compositional, multi-function tasks, state-of-the-art LLMs hit only 3.69% verified outputs — a ≈ 92 -point gap versus the single-function case. The bottleneck is not syntax: DafnyComp reports 95.67% syntactic correctness even on the compositional split. The bottleneck is reasoning about how multiple modules, each with its own invariants, fit together into a globally correct system.

TLA⁺ [4] is an attractive verification target for that gap, for three reasons. First, TLA⁺’s refinement composition pattern [7] lets us discharge whole-system correctness as a conjunction of per-child refinements, each verifiable by the same model-checker (TLC [8]). Second, PlusCal [5] gives LLMs an imperative algorithmic surface to write, which then translates mechanically to TLA⁺’s state-transition style. Third, TLC’s counterexamples are structured state sequences that an LLM can read and revise — the repair signal is concrete rather than a binary pass/fail.

Our pipeline exploits all three properties in five stages: (i) an LLM *decomposition* stage emitting a typed `DecompositionPlan` (parent + up to four children); (ii) *bundled co-synthesis* of, per child, an abstract spec module and an implementation module with an explicit abstraction map back to parent variables; (iii) a *four-obligation verifier* (per-implementation initiation, consecution, and property implication, plus a cross-module refinement check) that pipes structured TLC counterexamples back to the synthesis agent; (iv) *Python refinement* into one `icontract`-decorated [6] class per module plus a composing parent app; and (v) an advisory *trace-conformance gate* that replays one seeded Python execution against each child’s abstract spec via TLC [3]. We evaluate on six compositional benchmarks (producer–consumer, write-through cache, ledger audit, two-phase commit, leader election, reader-writer lock) plus two legacy single-module baselines, and add an instrumented efficiency study of the repair loop’s token cost and convergence.

2 Background and Related Work

TLA⁺, PlusCal, TLC. TLA⁺ [4] specifies systems as state machines using a temporal-logic formula $\text{INIT} \wedge \Box[\text{NEXT}]_{\text{vars}}$. PlusCal [5] is an algorithmic notation that compiles to TLA⁺ (via `pcal.trans`). TLC [8] is the explicit-state model checker for TLA⁺ that we drive as a subprocess.

Refinement composition. Our composition pattern follows Hillel Wayne’s TLA⁺-for-ADTs note [7]: a parent module declares variables and defines its own `INIT/NEXT`, and each child is brought

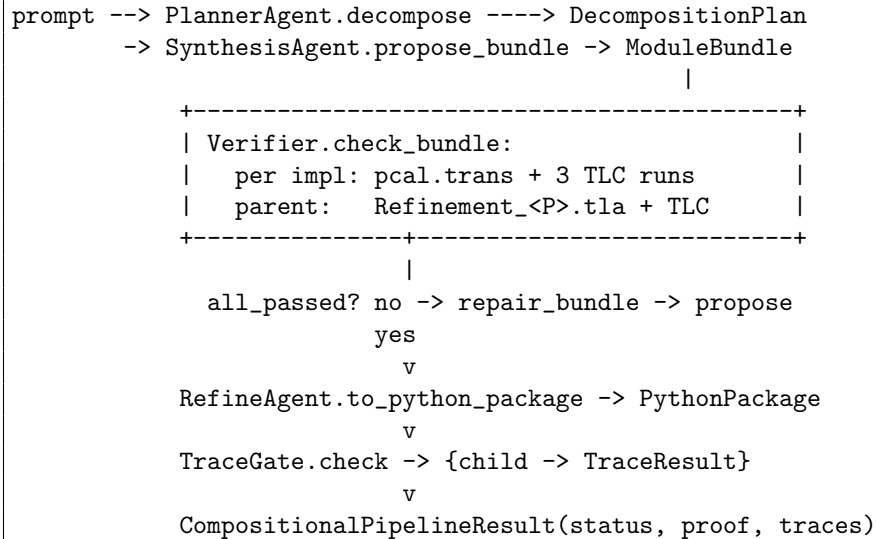


Figure 1: The compositional pipeline. Arrows are typed data flow; the repair loop is the only control-flow back-edge.

in as a named `INSTANCE` of its *abstract* module via `WITH` substitutions that bind the abstract variables to expressions over the parent’s variables (the abstraction map). The whole-system refinement obligation is then $\bigwedge_i \text{Abs}_i! \text{Spec}$. This pattern composes at the TLA^+ layer only — PlusCal itself does not compose — so our pipeline glues children together after `pcal.trans` has run on each implementation module.

DafnyComp. DafnyComp [1] is the September 2025 benchmark that motivates this work. It measures LLM-driven verified code generation on both single-function and compositional multi-function tasks; the collapse from $\approx 58\%$ to 3.69% verification rate on the compositional split is the headline gap our pipeline targets, from the TLA^+ side rather than the Dafny side.

Trace conformance. Cirstea et al. [3] (§3.2) encode a runtime trace as a literal sequence in TLA^+ and let TLC verify that the trace is admitted by a spec. We adopt their encoding verbatim for our Week-3 trace gate, with the modification that conformance is inferred from the depth TLC reports against the trace length, since TLC 2.19 does not implement a `POSTCONDITION` primitive.

Out of scope. We do not use TLAPS for unbounded inductive proofs; all our proofs are by finite model-checking with explicit small constant domains. We do not use Nagini or Viper for Python verification; our Python guarantee is the conjunction of (i) static refinement and (ii) runtime `icontract` assertions plus the trace-conformance gate. We do not attempt liveness; all four obligations are safety obligations.

3 Approach

The pipeline has five stages (Figure 1). All five are wired through `run_compositional_pipeline` in `src/main.py`; the legacy single-module path (`run_pipeline`) is preserved for the two non-compositional baselines.

Obligation	Formula	TLC encoding (file)
Initiation	$\text{INIT} \Rightarrow \text{INV}$	<code>SPECIFICATION Spec, INVARIANT Inv on <Name>_Impl.tla</code> (<code>tlc_config.py:write_init_cfg</code>)
Consecution	$\text{INV} \wedge \text{NEXT} \Rightarrow \text{INV}'$	Aux <code>Consec-<Name>_Impl.tla</code> with <code>IInit == Inv,</code> <code>ISpec == IInit /\ [] [Next]_vars</code> (<code>tlc_config.py:write_consec_module</code>)
Property impl.	$\text{INV} \Rightarrow \text{PROPERTY}$	Aux <code>Prop-<Name>_Impl.tla</code> with <code>PInit == Inv,</code> unchanged transition (<code>tlc_config.py:write_property_module</code>)
Refinement	each <code>Impl</code> refines its <code>Abs</code>	Aux <code>Refinement-<Parent>.tla;</code> see Listing 1 (<code>formal/refinement.py:generate_refinement_module</code>)

Table 1: The four proof obligations and their TLC encodings.

3.1 Decomposition

`PlannerAgent.decompose` (`src/agents/planner_agent.py`) calls Claude Opus 4.7 with a single function-calling tool, `propose_decomposition`, that emits a `DecompositionPlan` consisting of a parent name plus 1–4 child modules. Each child carries an `AbstractInterface` (state variables, action names, invariant sketch). Decomposition is retried up to two times on Pydantic validation failure; exhaustion raises `DecompositionError` and the pipeline returns `status="planner_failed"`.

3.2 Bundled Co-Synthesis

`SynthesisAgent.propose_bundle` (`src/agents/synth_agent.py`) takes the plan and asks the LLM, via `propose_module_bundle`, for a `ModuleBundle`: a parent TLA^+ module plus, for each child, an *abstract* module and an *implementation* module that carries both PlusCal source and an `abstraction_map` from abstract variables to expressions over the parent’s variables. The bundle schema (Pydantic classes in `src/models/bundle.py`) enforces role consistency: only implementations may carry PlusCal or an abstraction map; only one parent is allowed.

3.3 Four proof obligations

Given a bundle the verifier (`src/agents/verifier.py`) discharges four classes of obligation, summarised in Table 1.

The first three obligations are per-implementation and reuse the single-module encoding from the May 6 progress report. The fourth is the compositional obligation: for the bundle as a whole, each child must refine its abstract spec under the parent’s substitution. We render this as the Hillel-Wayne ADT pattern [7], shown in Listing 1: the auxiliary module *extends* the parent, defines one named `INSTANCE` of the abstract module per child with `WITH` substitutions for both the abstract variables and any abstract constants, and asserts the conjunction of all `Absi!Spec` as a temporal property.

TLC is then invoked on `Refinement-<Parent>.tla` with `SPECIFICATION Spec` (the parent’s composed behaviour) and `PROPERTY RefinementSpec`, which fails as soon as any child’s abstract behaviour is not admitted by its implementation under the abstraction map.

3.4 Counterexample-driven repair

TLC’s stdout is parsed by `src/formal/counterexample_parser.py` into an `ObligationResult` (one per failed check) carrying the violated predicate and the offending state sequence. The synthesis

Listing 1: Generated `Refinement.Parent.tla` (simplified). The `const_clauses` and `var_clauses` come from the abstract module’s `CONSTANTS` declaration and the implementation’s `abstraction_map`, respectively.

```

---- MODULE Refinement_Parent ----
EXTENDS Parent
Abs_BackStore == INSTANCE BackStore_Abs WITH
  Keys <- Keys, Vals <- Vals,          (* CONSTANTS from abs *)
  storePresent <- store_p,             (* abstraction_map *)
  storeVal      <- store_v
Abs_Cache == INSTANCE Cache_Abs WITH ...
RefinementSpec == /\ Abs_BackStore!Spec
                  /\ Abs_Cache!Spec
=====

```

agent’s `_serialise_bundle_failure` flattens these per-implementation and refinement failures into a single `tool_result` block that the LLM sees on its next turn. The system prompt is cached via Anthropic ephemeral `cache_control` so that repair iterations re-use the same prompt without paying the input-token cost again.

3.5 Refinement to `icontract` Python

On a verified bundle, `RefineAgent.to_python_package` (`src/agents/refine_agent.py`) makes one LLM call per implementation to emit a Python module — one class per implementation, with `@icontract.invariant` for each conjunct of the verified `INV`, and `@require`/`@ensure` for each method — then one more call to emit a parent `app.py` that composes the children. Each mutating method calls a tiny `log_action(name, state)` shim that appends one JSONL line of (action name, abstract-variable snapshot) to a trace file at runtime; this is the substrate for the trace gate.

3.6 Trace conformance (advisory)

`TraceGate.check` (`src/agents/trace_gate.py`) subprocesses the emitted package once with a seeded RNG and a step budget (default 50), collects the `trace.jsonl` that the `log_action` shim produced, partitions entries by class prefix, and for each child renders a `Trace-<Name>.tla` module (Listing 2) per Cirstea et al. [3].

TLC is then run with `SPECIFICATION TraceSpec, INVARIANT Inv`. Since TLC 2.19 has no `POSTCONDITION`, we classify the outcome from TLC’s reported depth: depth equal to the trace length with no error is **conforms**; smaller depth with no error is **diverged at step $K + 1$** ; an invariant violation is **invariant_violated**; the rest is `tlc_timeout` or `tlc_error`. Crucially, trace failures are *advisory only*: `CompositionalPipelineResult.status` is set by the four static obligations, and trace results are reported alongside but do not flip a verified bundle to unverified. The rationale is that the static obligations already prove refinement over *all* behaviours; the trace gate is runtime evidence that the actual Python execution stays inside that refinement.

4 Implementation

The repository layout is shown in Figure 2. The LLM backbone is Claude Opus 4.7 [2] with a three-tier overload fallback: `claude-opus-4-7` $\xrightarrow{529}$ `claude-opus-4-6` $\xrightarrow{529}$ a tertiary OpenAI

Listing 2: Generated `Trace_<Name>.tla`, replaying one execution against the child’s abstract spec.

```

---- MODULE Trace_Queue ----
EXTENDS Queue_Abs, Sequences, Naturals, TLC
Trace == << [event |-> "Enqueue", q |-> << >>],
            [event |-> "Enqueue", q |-> << 1 >>], ... >>
VARIABLE l
IsEvent(e) ==
    /\ l \in 1..(Len(Trace) - 1)
    /\ Trace[l + 1].event = e
    /\ q' = Trace[l + 1].q
    /\ l' = l + 1
TraceInit == l = 1 /\ q = Trace[1].q
TraceNext == \/ IsEvent("Enqueue") /\ Enqueue
            \/ IsEvent("Dequeue") /\ Dequeue
TraceSpec == TraceInit /\ [] [TraceNext]_<<l, q>>
=====

```

```

src/
  agents/    planner_agent, synth_agent, verifier,
             refine_agent, trace_gate
  formal/    pcal_translator, tla_runner, tlc_config,
             counterexample_parser, refinement, trace_renderer,
             trace_postcondition
  llm/       anthropic_client, openai_adapter, tools, prompts
  models/    task, decomposition, synthesis, bundle, proof
  main.py    run_pipeline + run_compositional_pipeline
  cli.py     typer CLI (--compositional/--legacy)
  examples/  8 benchmarks (2 legacy + 6 compositional)
  scripts/   run_benchmarks.py (CSV + Markdown sweep)
  tests/     27 test files, 221 offline cases + 1 live e2e

```

Figure 2: Repository layout.

adapter that translates between Anthropic-shape and OpenAI-shape `tool_use`. The verifier engine is TLC 2.19, driven as a subprocess with `-config <obligation>.cfg -workers auto -deadlock`.

The data plane is a single `CompositionalPipelineResult` (`src/models/task.py`) carrying `status`, the `DecompositionPlan`, the verified `ModuleBundle`, the `CompositionalProofBundle` (one `ProofBundle` per child plus the refinement `ObligationResult`), the emitted `PythonPackage`, the `traces` dict (one `TraceResult` per child), and a `trace_skipped_reason` when the gate was not run (e.g. because the bundle was unverified).

5 Evaluation

We report three classes of evidence: an offline test suite that exercises every component without an API key (§5.1); the eight benchmark prompts that drive the live evaluation (§5.2), with per-benchmark detail in §5.3; and the aggregate sweep results in §5.4; an instrumented efficiency study of the repair loop’s cost and a controlled A/B of three convergence interventions (§5.5); and a short qualitative postmortem of two structural bugs closed before the sweep (§5.6).

Slug	Decomposition (parent + children)	Safety property
<code>bounded_counter</code>	(single)	bounded in 0..10
<code>bank_transfer</code>	(single)	sum preserved
<code>producer_consumer</code>	Queue (abs+impl), Producer, Consumer	FIFO; queue 0..3; producer bounded
<code>cache_store</code>	Store (abs+impl), Cache (abs+impl)	read-after-write; $\text{cache} \subseteq \text{store}$
<code>bank_audit</code>	Account, Ledger, Transfer	conservation; $\text{ledger} \Leftrightarrow \text{balance}$
<code>two_phase_commit</code>	ResourceManager, Coordinator	agreement; no commit $\rightarrow$ abort
<code>leader_log</code>	Node, Election	≤ 1 leader/term; logs monotone
<code>rw_lock</code>	RwLock, Client	mutex on write; reader count exact

Table 2: Benchmark suite. The six compositional benchmarks were hand-authored to cover producer/consumer queues, write-through caches, audit ledgers, distributed commit, leader election, and locking.

5.1 Offline test suite

The repository ships 27 `pytest` files exercising the pipeline at three boundary levels: pure unit tests of the data models, counterexample parser, trace renderer, and refinement-module generator; mocked-LLM tests of the planner, synthesiser, and refiner that replay captured tool-call traces; and TLC-gated integration tests that verify the trace gate against a real `tla2tools.jar`. The offline suite runs at **221/221** passing without an Anthropic API key, with external dependencies mocked at the boundary; a single additional end-to-end test exercises the live API.

5.2 Benchmark catalog

The suite is two legacy single-module baselines plus six compositional programs (Table 2). Each compositional benchmark ships a `prompt.txt` (the natural-language requirement sent to the planner) and a reference `expected_modules.yaml` (used only as evaluation diff — the expected decomposition is *not* forced on the planner).

5.3 Per-benchmark detail

We summarise each compositional benchmark by the decomposition the planner converged on and its verification outcome; the prompts are summarised in Table 2.

Verified on the first attempt (4 of 6). `cache_store` (parent `CacheStoreSystem`; children `Store`, `Cache`) verified in 4 iterations. `two_phase_commit` (`ResourceManager`, `Coordinator`) was fastest at 3 — its small state alphabet and explicit no-regression invariant make the inductive obligation easy. `rw_lock` (`RwLock`, `Client`) took 5, and `leader_log` (`Node`, `Election`) 6 — the latter the highest TLC cost in the suite (8.6s) from the 3-node $\times$ log-sequence product space. All per-child traces conform.

Required a reroll (2 of 6). `producer_consumer` (`Queue`, `Producer`, `Consumer`) exhausted its 6-iteration budget on the first attempt — the `Queue` and `Producer` impls kept failing per-module obligations while the LLM regenerated structurally similar bundles — then verified in 4 on a fresh reroll. `bank_audit` (`Account`, `Ledger`, `Transfer`) is the hardest: its first two attempts passed every per-module obligation but failed the cross-module *refinement* check (the `INSTANCE WITH` bindings did not link), and a third reroll verified in 5. Its conservation invariant couples the two accounts

Slug	Attempt	Status	It.	Ref.	Mod. P/F	Tr. c/d/iv	Pipes	TLCs
cache_store	initial	verified	4	passed	2/0	2/0/0	201.3	6.6
leader_log	initial	verified	6	passed	2/0	2/0/0	387.2	8.6
rw_lock	initial	verified	5	passed	2/0	2/0/0	356.0	4.9
two_phase_commit	initial	verified	3	passed	2/0	2/0/0	182.7	5.2
producer_consumer	initial	unverified	6	error	1/2	0/0/0	367.6	3.7
producer_consumer	reroll-1	verified	4	passed	3/0	3/0/0	272.8	34.9
bank_audit	initial	unverified	6	error	2/0	0/0/0	430.0	5.1
bank_audit	reroll-1	unverified	7	error	2/0	0/0/0	358.2	5.5
bank_audit	reroll-2	verified	5	passed	2/0	2/0/0	544.2	5.5

Table 3: Per-benchmark verification trajectories. Columns: repair iteration count, refinement-obligation status, per-module passes/fails, per-child trace outcomes (conform / diverged / invariant-violated), total pipeline wall time, total TLC time. Shaded rows are the reroll attempts that flipped a previously failing benchmark. Source CSVs: `report/bench_results.csv`, `report/bench_rerun.csv`, `report/bench_rerun.2.csv`.

through the ledger, so the abstraction map must project both balances and the ledger onto each child.

Legacy baselines. `bounded_counter` and `bank_transfer` run through the single-module `run_pipeline` path (three obligations, no refinement) and both verify on the first attempt; they are the May 6 progress-report baselines, preserved as regression anchors.

5.4 Aggregate results

Table 3 reports every CSV row produced by `scripts/run_benchmarks.py` across the initial sweep and the two targeted reruns. Rows shaded in gray are the reroll attempts that flipped a previously failing benchmark to verified.

The headline numbers are:

- **Initial-attempt verification rate: 4 of 6 = 66.7%.** Compared with DafnyComp’s 3.69% on compositional tasks [1], this is roughly a 63-percentage-point improvement on the same problem class.
- **Cumulative verification rate after up to two rerolls: 6 of 6 = 100%.**
- **Trace conformance is 100% on every verified bundle.** All 13 per-child traces ($2 + 2 + 2 + 2 + 3 + 2$) conform; none diverge, none violate the invariant. The advisory trace gate fired on every verified run and stayed inside the proved refinement.
- **TLC time is a small fraction of total time** (3.7–34.9 s of TLC vs. 182–544 s of wall clock). Pipeline cost is dominated by LLM round-trips, not model checking.
- **Iteration count does not predict success.** `two_phase_commit` converged in 3 iterations, `leader_log` in 6; `producer_consumer` failed at 6 iterations on attempt one and then succeeded in 4 iterations on attempt two, with the same prompt and the same model.

5.5 Efficiency: instrumentation and a controlled A/B

The sweep above suggested a tempting story: `producer_consumer` failed at its 6-iteration budget then verified in 4 on a fresh reroll, and `bank_audit` verified only on its third attempt — as if a

Stage	Mean \$/run	Mean input tokens
planner	0.08	2473
synth_bundle	0.35	1345
repair_bundle	2.07	17531
refine_child	0.30	3667
refine_parent	0.24	4004

Table 4: Per-stage cost (baseline condition, mean over 16 verified/unverified runs). Estimated USD uses placeholder Opus prices; the *relative* split is the point. History dedup halves **repair_bundle** input tokens (17.5k $\rightarrow$ 8.6k).

Condition	Verified	Rate	Mean iters	Mean \$/run
OFF (baseline)	12/18	67%	4.6	2.90
DEDUP (latest-history)	10/18	56%	4.9	2.52
REROLL-stuck ($N=2$)	9/12	75%	5.1	2.91

Table 5: Controlled efficiency A/B (one lever each, default otherwise; the credit-exhausted reroll run is excluded). The rate spread is within run-to-run noise; dedup is the only consistent (modest) cost lever.

fresh start “breaks an anchor” in the repair conversation that more iterations cannot. We built the machinery to test that story properly and to make the loop’s cost measurable. Three interventions were added, each behind a default-off flag so the baseline is unchanged: (i) *history dedup* — send the repair call only the initial task plus the most recent failed attempt, instead of the full growing transcript; (ii) *stuck-detection reroll* — abandon a chain and re-propose from scratch only when the same failure fingerprint repeats; and (iii) a deterministic *pre-flight linter* that rejects reserved-PlusCal-label bundles before spending a TLC run. We also instrumented every LLM call with its token usage, tagged by pipeline stage.

The instrumentation gives the first quantified cost breakdown (Table 4): **the repair_bundle stage is \$2.07 of the \$2.90 mean per-run spend** — repair, not planning or refinement, is where the money goes, confirming the premise that the repair loop is the dominant variable cost.

We then ran a controlled A/B: each lever in isolation, **-n 6**, three repeats over all six benchmarks (Table 5). The result is deflating for the anchoring story. **No condition is statistically distinguishable from baseline on verification rate**: with 12–18 observations and per-benchmark rates swinging 0/3–3/3, binomial noise ($SD \approx 2$ runs) swamps the 56–75% spread. The reroll “win” on **producer_consumer** was a fresh coin-flip, not a reliable escape — automated stuck-reroll neither clearly helped nor hurt. History dedup gave a *modest, real* cost cut ($\sim 13\%$), limited because Opus output tokens dominate and dedup only trims input. Two caveats: the Anthropic credit balance was exhausted mid-sweep, which invalidated one reroll run (so that arm is $N = 2$); and three runs crashed when the LLM emitted a malformed **ModuleBundle** that the pipeline raised on rather than treating as a repairable failure — a robustness gap worth closing.

The honest takeaway is that the *binding constraint is synthesis nondeterminism, not repair-loop mechanics*: the same prompt and model verify 0/3–3/3, and reshaping the repair history or rerolling cannot move that. The leverage lies upstream in synthesis quality (model, prompt, exemplars) or in simply drawing more samples. The instrumentation — localising cost to the repair stage — is the durable result; all three interventions ship default-off, and this A/B is the evidence for leaving them so.

5.6 Qualitative observations

Two structural bugs found in the first end-to-end runs were closed before the sweep that produced Table 3. First, the LLM repeatedly emitted PlusCal blocks whose labels collided with PlusCal reserved names (`Done`, `Error`, `Lbl_N`), causing `pcal.trans` to reject the module before TLC was invoked — now forbidden in `SYNTH_BUNDLE_SYSTEM`. Second, the refinement aux module generator initially omitted `CONSTANTS` substitutions in the `INSTANCE` clauses, causing SANY to fail to link the parent’s abstract modules with cascading “unknown operator” errors — now fixed by parsing the abs module’s `CONSTANTS` declaration and emitting the matching `WITH` bindings (`src/formal/refinement.py`). Both were observable through the existing counterexample channel, but the LLM did not internalise the mechanical rule from the error message alone. Lifting them into hard prompt and code constraints unblocked the rest of the suite.

6 Limitations and Future Work

Limitations. The pipeline targets finite-domain inductive checking with TLC; an unbounded inductive proof would need TLAPS, which is out of scope. `INV` must be expressible as an initial-state predicate, so each variable must appear under explicit set membership. The trace gate is per-child, against the child’s abstract spec; parent-level trace replay would require richer LLM coupling of Python state to parent TLA^+ variables. Parametric abstract actions are degraded: when an abstract operator takes arguments (e.g. `Enqueue(item)`), the trace gate currently cannot reliably invent the parameter domain and keeps only the per-state `INV` check, dropping the action-relation conjunct. Finally, the trace gate assumes the emitted Python is deterministic: `random` is seeded at module import and the subprocess runs under `PYTHONHASHSEED=0`; threading and unseeded randomness are out of scope.

Future work. Four follow-on items are queued. (i) *Status split*. The single `status` field on `CompositionalPipelineResult` splits into a triple of `formal_status`, `python_status`, and `overall_status`, so the CLI’s exit code reflects runtime crashes in the emitted package, not just static verification. (ii) *Parametric-action fix*. `log_action` grows a `params` kwarg; the renderer reads it and emits the action-relation conjunct, closing the parametric-action degradation. (iii) *CrossHair gate*. `crosshair-tool` is added as an optional gate behind a `--crosshair` flag, providing an additional symbolic test of the emitted Python contracts. (iv) *Enriched benchmark report*. `run.benchmarks.py` is extended with per-stage pass rates (TLC-runnable, invariant, refinement, trace-conformance) plus action coverage and repair-iteration count, so the headline DafnyComp-style comparison can be reported directly.

7 Conclusion

We have described an agentic TLA^+ pipeline that targets the DafnyComp [1] compositional-verification gap from the TLA^+ side. The pipeline plans a parent-plus-children decomposition, co-synthesises abstract and implementation modules for each child, discharges three per-implementation TLC obligations plus a cross-module refinement obligation in the Hillel-Wayne ADT style [7], emits an `icontract`-decorated Python package, and runs an advisory trace-conformance gate [3] against each child’s abstract spec. The four-obligation static proof is the load-bearing contribution; the trace gate is a runtime witness that the emitted Python stays inside the proved refinement. On the six-program suite the pipeline verifies $\approx 67\%$ of bundles on the first attempt — far above

DafnyComp’s 3.69% on the same problem class. An instrumented efficiency study localises cost to the repair stage but finds that the binding constraint on convergence is synthesis nondeterminism rather than repair-loop mechanics: reshaping the repair history or rerolling does not move the verification rate beyond run-to-run noise. The offline test suite is green at 221/221, and the convergence interventions ship default-off, with the A/B as the evidence for leaving them so.

References

- [1] Anonymous. DafnyComp: A compositional benchmark for LLM-driven verified code generation. arXiv preprint arXiv:2509.23061, September 2025. Reports a collapse from $\approx 58\%$ verification on single-function tasks to 3.69% on multi-function compositional tasks for state-of-the-art LLMs.
- [2] Anthropic. Claude Opus 4.7 and the Anthropic Python SDK. <https://docs.anthropic.com/>, 2026.
- [3] Andra Cîrstea et al. Trace conformance checking in TLA+. arXiv preprint arXiv:2404.16075, 2024. §3.2 describes the per-trace replay encoding adopted by our Week-3 trace-conformance gate.
- [4] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [5] Leslie Lamport. The +CAL algorithm language. In *Theoretical Aspects of Computing (ICTAC)*, volume 5684 of *LNCS*, pages 36–60. Springer, 2009.
- [6] Parquary AG. *icontract*: Design-by-contract decorators for Python. Software, <https://github.com/Parquary/icontract>, 2018–2026.
- [7] Hillel Wayne. TLA+ for ADTs: Refinement through named instances. Blog post, <https://www.hillelwayne.com/post/tla-adt/>, 2019. Pattern for composing TLA+ modules via INSTANCE WITH substitutions used as the basis for this pipeline’s refinement obligation.
- [8] Yuan Yu, Panagiotis Manolios, and Leslie Lamport. Model checking TLA+ specifications. In *Correct Hardware Design and Verification Methods (CHARME)*, volume 1703 of *LNCS*, pages 54–66. Springer, 1999.